



COMSATS University Islamabad

Attock Campus

Department of Computer Science

Course	Information Security
Instructor	Mam Ambreen Gull
Assignment No.	03

Student Details

Registration No.	Name
FA24-BSE-015	Areesha Imran

Task 1: Basic Blockchain Implementation:

1.1 Overview

A blockchain is an ordered, linked list of blocks where each block holds a cryptographic hash of the block before it. This makes the chain tamper-evident: altering any block invalidates every block after it. Python's built-in `hashlib.sha256` is used to compute all hashes.

1.2 Block Class

Constructor — `__init__`

```
class Block: def __init__(self, index, data,
previous_hash):

self.index = index self.timestamp = time.strftime('%Y-%m-%d %H:%M:%S',
time.localtime()) self.data = data self.previous_hash = previous_hash
self.hash = self.calculate_hash()
```

Field	Explanation
self.index	Position of this block in the chain (0 = genesis block).
self.timestamp	Creation time recorded using the system clock.
self.data	Payload stored in the block (e.g., transaction string).
self.previous_hash	SHA-256 hash of the preceding block — the cryptographic link.
self.hash	This block's own fingerprint, computed by <code>calculate_hash()</code> .

`calculate_hash()` Method

```
def calculate_hash(self):
block_contents = ( str(self.index) + str(self.timestamp) +
str(self.data) + str(self.previous_hash) ) return
hashlib.sha256(block_contents.encode()).hexdigest()
```

All four fields are concatenated into one string. `encode()` converts it to bytes (SHA-256 requires bytes). `hexdigest()` returns the 64-character hex digest. A single character change anywhere produces a completely different output — this is the avalanche effect.

1.3 Blockchain Class

Genesis Block

```
def create_genesis_block(self):
genesis = Block(index=0, data='Genesis Block', previous_hash='0')
self.chain.append(genesis)
```

Block #0 has no predecessor so its `previous_hash` is the literal string '0'. Every subsequent block links back through an unbroken hash chain to this anchor.

`add_block()` Method

```
def add_block(self, data):
```

```
previous_block = self.get_latest_block() new_block =  
Block( index = len(self.chain), data = data,  
previous_hash = previous_block.hash )  
self.chain.append(new_block)
```

The new block receives the current tip's hash as its `previous_hash`. This is the core linking mechanism — each block knows exactly which block came before it.

`is_chain_valid()` — Chain Validation

```
def is_chain_valid(self):  
    for i in range(1, len(self.chain)):  
        current = self.chain[i]  
        previous = self.chain[i - 1]  
  
        # Rule 1: stored hash must equal freshly computed  
        if current.hash != current.calculate_hash():  
            return False  
  
        # Rule 2: block must point to correct previous hash  
        if current.previous_hash != previous.hash:  
            return False  
    return True
```

Two independent checks are applied to every block (genesis is skipped). **Rule 1** detects data mutation within a block. **Rule 2** detects insertion, removal, or reordering of blocks. Both must pass for the chain to be considered valid.

`tamper_block()` — Tamper Demonstration

```
def tamper_block(self, index, new_data):  
    if 0 < index < len(self.chain):  
        self.chain[index].data = new_data # hash  
        is intentionally NOT recalculated
```

Data is overwritten directly without updating the stored hash. When `is_chain_valid()` is called next, Rule 1 fails immediately — the stored hash no longer matches the recomputed hash of the modified content, proving any tampering is detectable.

1.4 Program Output

The screenshots below show the actual terminal output when `task1_blockchain.py` is executed.

```
#####
#      TASK 1: BASIC BLOCKCHAIN IMPLEMENTATION      #
#####

--- Adding blocks to the chain ---
[+] Block #1 added successfully.
[+] Block #2 added successfully.
[+] Block #3 added successfully.
[+] Block #4 added successfully.

BLOCKCHAIN - FULL CHAIN VIEW

Block #0
Timestamp      : 2026-05-02 20:41:33
Data           : Genesis Block
Previous Hash: 0
Hash           : 24813adfdb4fdb35706bf1a24632dd3d05adb8debb8df3c3c40f3c6280048a73

Block #1
Timestamp      : 2026-05-02 20:41:33
Data           : Alice pays Bob 50 coins
Previous Hash: 24813adfdb4fdb35706bf1a24632dd3d05adb8debb8df3c3c40f3c6280048a73
Hash           : 61258f7df518bb9283f147aec1c0b79490407eb255062871d3fe5eb4a2f939a9

Block #2
Timestamp      : 2026-05-02 20:41:33
Data           : Bob pays Carol 20 coins
Previous Hash: 61258f7df518bb9283f147aec1c0b79490407eb255062871d3fe5eb4a2f939a9
Hash           : 507d3a5a2969127a8bf6a24c1f0e912384ef72974e053da645eca43d1493730c

Block #3
Timestamp      : 2026-05-02 20:41:33
Data           : Carol pays Dave 10 coins
Previous Hash: 507d3a5a2969127a8bf6a24c1f0e912384ef72974e053da645eca43d1493730c
Hash           : f3d7900936c35b253e3f61ceef5b91fd4c02d4da56126d4d9f32fc17f714d192

Block #4
Timestamp      : 2026-05-02 20:41:33
Data           : Dave pays Eve 5 coins
Previous Hash: f3d7900936c35b253e3f61ceef5b91fd4c02d4da56126d4d9f32fc17f714d192
Hash           : 34a6c078224cb57e308c9183421366ed5a059128ef42c31f04ae5802cc8396d6

--- Integrity Check (Before Tampering) ---
Chain valid: True

[TAMPER] Changing Block #2 data to: 'Bob pays Carol 9999 coins'

--- Integrity Check (After Tampering) ---
[!] Block #2 hash is invalid - data may have been tampered with.
Chain valid: False

Explanation: Changing Block #2 data invalidates its hash.
All subsequent blocks that reference it are also compromised.
```

Screenshot 1

Block #3

```
Timestamp      : 2026-05-02 20:41:33
Data           : Carol pays Dave 10 coins
Previous Hash: 507d3a5a2969127a8bf6a24c1f0e912384ef72974e053da645eca43d1493730c
Hash           : f3d7900936c35b253e3f61ceef5b91fd4c02d4da56126d4d9f32fc17f714d192
```

Block #4

```
Timestamp      : 2026-05-02 20:41:33
Data           : Dave pays Eve 5 coins
Previous Hash: f3d7900936c35b253e3f61ceef5b91fd4c02d4da56126d4d9f32fc17f714d192
Hash           : 34a6c078224cb57e308c9183421366ed5a059128ef42c31f04ae5802cc8396d6
```

--- Integrity Check (Before Tampering) ---

Chain valid: True

[TAMPER] Changing Block #2 data to: 'Bob pays Carol 9999 coins'

--- Integrity Check (After Tampering) ---

[!] Block #2 hash is invalid - data may have been tampered with.
Chain valid: False

Explanation: Changing Block #2 data invalidates its hash.
All subsequent blocks that reference it are also compromised.

Screenshot 2

Task 2: Elliptic Curve Cryptography (ECC) Basics:

2.1 Overview

ECC operates over a finite field F_p (integers mod a prime p). The curve is defined by: $y^2 \equiv x^3 + ax + b \pmod{p}$. Points on the curve form a mathematical group. Security rests on the Elliptic Curve Discrete Logarithm Problem (ECDLP): given G and $Q = k \cdot G$, recovering k is computationally infeasible.

Field	Explanation
$p = 17$	Prime modulus — defines the finite field F_{17} .
$a=2, b=2$	Coefficients of curve $y^2 = x^3 + 2x + 2 \pmod{17}$.
$G = (5,1)$	Generator point verified to lie on the curve.
$d = 7$	Private key (chosen manually; in practice secret and random).
$Q = d \cdot G$	Public key — computed via scalar multiplication.

2.2 EllipticCurve Class

`is_on_curve()` — Point Validation

```
def is_on_curve(self, point):
    if point is None: return True # point at infinity is always valid
    x, y = point
    lhs = (y * y) % self.p # y^2 mod p
    rhs = (x**3 + self.a * x + self.b) % self.p # x^3+ax+b mod p
    return lhs == rhs
```

Verifies that a point satisfies the curve equation in the finite field. `None` represents the point at infinity — the additive identity of the group, which is always considered valid.

`mod_inverse()` — Modular Inverse via Fermat's Little Theorem

```
def mod_inverse(self, k):
    return pow(k, self.p - 2, self.p)
# Fermat: k^(p-2) mod p = k^(-1) mod p (valid when p is prime)
```

In modular arithmetic, division is replaced by multiplication by the modular inverse. Python's `pow(k, e, m)` computes this efficiently using fast modular exponentiation in $O(\log p)$ steps.

point_addition() — Adding Two Distinct Points

```
def point_addition(self, P, Q): if P is None: return Q
# O + Q = Q if Q is None: return P # P + O = P if P ==
Q: return self.point_doubling(P) if x1 == x2: return
None # inverses → point at infinity
# slope: lambda = (y2-y1) / (x2-x1) mod p
lam = (y2 - y1) * mod_inverse(x2 - x1) mod
p x3 = (lam^2 - x1 - x2) mod p y3 =
(lam*(x1 - x3) - y1) mod p return (x3, y3)
```

The slope between two distinct points is computed using modular division. Resulting coordinates are reduced mod p to stay within the finite field. Each special case (identity element, inverse points, equal points) is handled before the general formula. [point_doubling\(\) — Doubling a Point](#)

```
def point_doubling(self, P): if P
is None or y1 == 0: return None
# tangent slope: lambda = (3*x1^2 + a) / (2*y1) mod
p lam = (3*x1^2 + a) * mod_inverse(2*y1) mod p x3 =
(lam^2 - 2*x1) mod p y3 = (lam*(x1 - x3) - y1) mod p
return (x3, y3)
```

When adding a point to itself, the secant slope formula is undefined (numerator and denominator both zero). Instead, the tangent slope derived from the curve's derivative is used: $(3x^2 + a) / (2y)$.

scalar_multiply() — Double-and-Add Algorithm

```
def scalar_multiply(self, k, P):
result = None # point at infinity (additive
identity) addend = P while k:
if k & 1: # if current least-significant bit is 1
result = point_addition(result, addend) addend =
point_doubling(addend) # double every iteration
k >>= 1 # shift k right by 1 bit
return result
```

Processes each bit of k from LSB to MSB. When the bit is 1 the current addend is added to the result; the addend is doubled every iteration regardless. This requires only $O(\log k)$ operations — making large-key scalar multiplication practical.

2.3 ECCKeyPair Class — Key Generation

```
class ECCKeyPair: def __init__(self,
curve, G, private_key):
self.private_key = private_key # d — kept secret self.public_key
= curve.scalar_multiply(private_key, G) # Q = d*G
```

The private key d is a secret integer. The public key Q is computed as $d \cdot G$ using scalar multiplication. The security guarantee: even knowing G and $Q = d \cdot G$ publicly, recovering d (the private key) requires solving ECDLP — computationally infeasible for large parameters.

2.4 ECElGamal Class — Encryption & Decryption

encrypt() Method

```
def encrypt(self, m, r): M = self.encode_message(m) #
encode m as curve point M
Q = self.key_pair.public_key # recipient's public key C1
= curve.scalar_multiply(r, G) # C1 = r * G rQ =
curve.scalar_multiply(r, Q) # shared secret component C2
= curve.point_addition(M, rQ) # C2 = M + r*Q return C1,
C2
```

Message m is first encoded as a point M on the curve. An ephemeral key r generates two ciphertext components. **C1** is a 'key hint' that lets the recipient reconstruct the shared secret. **C2** is the blinded message.

Without the private key, $r \cdot Q$ cannot be computed so M cannot be isolated from $C2$. [decrypt\(\) Method](#)

```
def decrypt(self, C1, C2):
d = self.key_pair.private_key # private key S =
curve.scalar_multiply(d, C1) # S = d*C1 = d*r*G = r*Q neg_S =
self.negate_point(S) # -S = (x, -y mod p) M =
curve.point_addition(C2, neg_S) # M = C2 - S return
self.decode_message(M) # return x-coordinate
```

The recipient multiplies $C1$ by their private key d . Since $C1 = r \cdot G$, $d \cdot C1 = d \cdot r \cdot G = r \cdot (d \cdot G) = r \cdot Q$ — the same shared secret used during encryption. Subtracting it from $C2$ cancels out the blinding, leaving only the original message point M .

2.5 Program Output

The screenshot below shows the actual terminal output when `task2_ecc.py` is executed.

```
#####
#      TASK 2: ELLIPTIC CURVE CRYPTOGRAPHY (ECC)      #
#####

--- Curve Parameters ---
Equation : y^2 = x^3 + 2x + 2 (mod 17)
Generator: G = (5, 1)
G on curve: True

--- Point Addition ---
P = (5, 1)    on curve: True
Q = (6, 3)    on curve: True
P + Q = (10, 6) on curve: True

--- Point Doubling ---
2P = 2 * (5, 1) = (6, 3)    on curve: True

--- Key Generation ---
Private Key (d) : 7 (kept secret)
Public Key (Q) : (0, 6) (Q = d * G = 7 * (5, 1))
Q on curve      : True

--- EC-ElGamal Encryption & Decryption ---

Original Message (m) : 5
Ephemeral Key (r) : 3

Encoded Point (M) : (5, 1)
Ciphertext C1 = r*G : (10, 6)
Ciphertext C2 = M+rQ : (10, 6)

Decrypted Point : (5, 1)
Decrypted Message : 5

Success: True - Original '5' matches decrypted '5'
PS C:\Users\DCS-ATK>
```

Security Analysis

3.1 Security of the Blockchain (Task 1)

Field	Explanation
SHA-256 Collision Resistance	Finding two different inputs that produce the same 256-bit hash requires $\sim 2^{128}$ operations. Crafting a fraudulent block with a matching hash is computationally infeasible.
Chain Integrity	Each block stores its predecessor's hash. Altering any block breaks the hash link, causing all subsequent blocks to fail validation. An attacker must recompute every following block's hash.
Tamper Evidence	Demonstrated directly: changing Block #2's data invalidates its stored hash, instantly detected by <code>is_chain_valid()</code> .
Avalanche Effect	SHA-256 ensures even a 1-bit change in input produces $\sim 50\%$ different output bits. Minor edits cannot go undetected.

Limitations	This demo lacks proof-of-work, peer-to-peer distribution, and consensus. In Bitcoin, attackers must also redo proof-of-work for every modified block — exponentially more expensive.
-------------	--

3.2 Security of ECC / EC-EIGamal (Task 2)

Field	Explanation
ECDLP Hardness	Security rests on the Elliptic Curve Discrete Logarithm Problem. Given $Q = d \cdot G$ and G publicly, recovering d requires solving ECDLP. No polynomial-time classical algorithm is known. Best attacks run in $O(\sqrt{p})$ time.
Key Security	Private key $d=7$ is trivially small (educational only). Production ECC uses 256-bit keys (NIST P-256), giving ~ 128 -bit security equivalent to 3072-bit RSA.
Ephemeral Key r	r must never be reused across encryptions. Reusing r allows an attacker to subtract ciphertexts, cancel the shared secret, and recover plaintext. In practice, r is generated freshly using a CSPRNG for each message.
Semantic Security	EC-EIGamal is IND-CPA secure under the Computational Diffie-Hellman assumption when r is chosen uniformly at random. Ciphertexts reveal nothing about the plaintext to a computationally bounded adversary.
Small Parameters	$p=17$ is chosen for clarity. The group order is tiny — brute force is trivial. Real systems use standardised curves (secp256k1, Curve25519) designed to resist MOV reduction and anomalous curve attacks.

3.3 Comparison Summary

Property	Blockchain SHA-256	ECC EIGamal
Purpose	Tamper-evident data storage	Asymmetric encryption
Core Operation	SHA-256 one-way hash	Scalar point multiplication
Key Concept	Collision & pre-image resistance	ECDLP hardness
Reversibility	Irreversible (one-way)	Reversible with private key
Security Level	$\sim 2^{128}$ collision resistance	$\sim 2^{(p/2)}$ for ECDLP
Real-World Use	Bitcoin, certificates, HMAC	TLS, SSH, crypto wallets